ECS 170

Artificial Intelligence

Fall 2026

Course Project: Stage 3 Report

Team Information

Enter Your Team Name Here 12		
Student 1: Theresa Dinh	Student 1: 923017620	Student 1: thtdinh@ucdavis.edu
Student 2: Justin Spenker	Student 2: 921684754	Student 2: jdspenker@ucdavis.edu
Student 3: Andy Huang	Student 3: 924512302	Student 3: xifhuang@ucdavis.edu
Student 4: Carley Smith	Student 4: 921776992	Student 4: carsmi@ucdavis.edu
Student 5: Eugene Ferdinand Ogilvie	Student 5: 925245571	Student 5: efogilvie@ucdavis.edu

Section 1: Task Description

In this stage we build convolutional neural network (CNN) models used for image classification on 3 data sets. We first train a CNN model on MNIST data, 28x28 grayscale images of handwritten digits (10 classes: 0-9), the same dataset we trained the MLP model on in stage 2. Then, we train a CNN on the ORL data set, consisting of 10 grayscale images each of 40 different faces (40 classes), each image at size 112x92. Finally, we train a CNN on the CIFAR dataset, 32x32 color images (3 channels) of everyday objects (10 classes). After training the models on these three datasets and reporting baseline results, we alter the configurations of the model and assess how these changes affect performance by looking at the changes in accuracy, precision, recall and F1-scores.

Section 2: Model Description

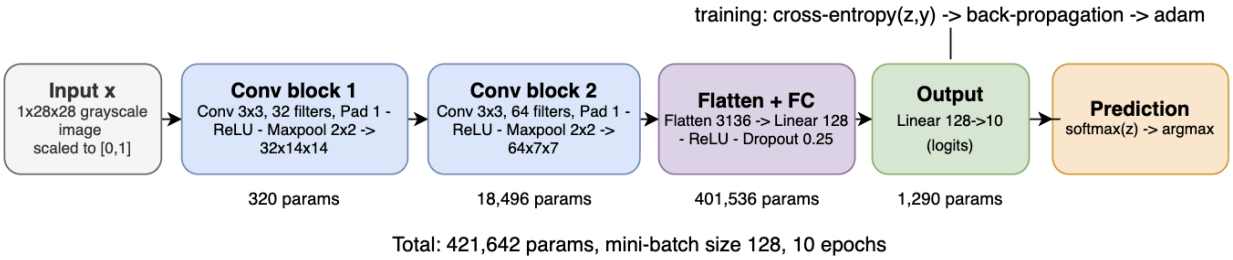


Figure 1: Baseline CNN for MNIST. The ORL and CIFAR models use the same design with different input sizes (Table 1)

The model is a convolutional neural network with two conv blocks, followed by a fully connected classifier (Figure 1). The input $x \in \mathbb{R}^{(1x28x28)}$ passes through **Conv(1→32, 3x3, pad 1) → ReLU → MaxPool(2x2)**, then **Conv(32→64, 3x3, pad 1) → ReLU → MaxPool(2x2)**, then **Flatten(3136) → Linear(3136→128) → ReLU → Dropout(0.25) → Linear(128→10)**. Convolution filters input to compute the weighted sum of a

3x3 patch, ensuring that equal weights will detect the same pattern anywhere in the image. Early layers of convolution detect edges which are later combined into feature shapes. Padding keeps height and width of each feature map constant, and ReLU avoids the saturation of sigmoid and tanh units. Max pooling picks out the strongest response in each 2x2 window, cutting the computation, and dropout (0.25) randomly zeros 25% of hidden units at each training step to reduce overfitting. The output layer produces one raw logit per class, and training loss is computed by applying log-softmax internally to logits using nn.`CrossEntropyLoss`. All weights are learned by back-propagation with mini-batch gradient descent using the Adam optimizer. ORL and CIFAR use the same architecture with different input sizes and number of classes (Table 1).

Property	MNIST	ORL	CIFAR
Input Shape	1x28x28	1x112x92	3x32x32
After Block 1	32x14x14	32x56x46	32x16x16
After Block 2	64x7x7	64x28x23	64x8x8
Flattened Size	3,136	41,216	4,096
Dropout	0.25	0.5	0.25
Output Classes	10	40	10
Conv Params (Both Blocks)	18,816	18,816	19,392
FC Hidden Params (→128)	401,536	5,275,776	524,416
Output Layer Params	1,290	5,160	1,290
Epochs/Batch Size	10/128	30/16	10/128
Train Size/Test Size	60,000/10,000	360/40	50,000/10,000
Total Params	421,642	5,299,752	545,098

Table 1: Baseline CNN for MNIST, ORL, and CIFAR

Section 3: Experiment Settings

3.1 Dataset Description

We use 3 datasets provided by the instructor for stage 3, one for each of the models: MNIST, ORL, and CIFAR-10. The MNIST dataset contains 70,000 grayscale images (28x28) of handwritten digits (0-9). The MNIST dataset is pre-partitioned into 60,000 training images and 10,000 testing images. The ORL dataset contains 400 grayscale images (112x92) of human faces, including 10 images for each of 40 individuals. The ORL dataset is pre-partitioned into 360 training images and 40 testing images. The CIFAR-10 dataset contains 60,000 color images (32x32x3), each depicting one of 10 different everyday objects. The CIFAR-10 dataset is pre-partitioned into 50,000 training images and 10,000 testing images.

3.2 Detailed Experimental Setups

The CNN models follow the architecture shown in Figure 1 (2 conv blocks → hidden FC → output) and layer sizes, batch sizes, epochs, and dropout shown in Table 1. All three inherit Adam optimizer, weight decay 0, and ReLU from the parent class Method_CNN. The models have learning rate 1e-3 and use cross-entropy loss on raw logits. PyTorch default initialization. All three datasets are preprocessed via pixel scaling to [0,1], ORL data is reduced from 3-channel to single-channel, and CIFAR data is kept as 3-channel. Random seed is fixed to 2 for NumPy and PyTorch for reproducibility. Settings across datasets differ in input size, number of classes, and training configuration. MNIST and CIFAR-10 each have 10

epochs, batch size 128, and dropout rate 0.25. ORL has 30 epochs, batch size 16, and dropout rate 0.5 (has only 360 images and needs more regularization). Model run on Python 3.13, PyTorch 2.5.1, scikit-learn 1.7.2, CPU (8 cores).

3.3 Evaluation Metrics

In the experiment, we will use Accuracy, Precision, Recall, F1 score as our evaluation metrics. Accuracy is the fraction of test instances whose predicted label equals the true label. For each class c, precision = TP/(TP+FP) (how many images predicted as c really are c), recall = TP/(TP+FN) (how many images of c are found), and F1 = 2·P·R/(P+R), their harmonic mean. Since the CNN models for MNIST, ORL, and CIFAR have 10, 40, and 10 output classes, respectively, we take the three averages: **micro average**, (uses the sum of all TP, FN, and FP over all classes), **weighted average** (mean weighted by number of test instances in each class), and **macro average** (unweighted mean over all classes). Since all three models use single label multi-class data, micro precision = micro recall = micro F1 (Table 2).

3.4 Source Code

Github: https://github.com/JSpenk/ECS170

The code follows the course template (abstract classes in `local_code/base_class/`). The Stage 3 files are:

- `local_code/stage_3_code/Dataset_Loader.py` – read the three pickle files (MNIST, ORL, and CIFAR) into features X with shape (n, channels, height, width), scaled to [0,1].
- `Method_CNN.py` – configurable convolutional neural network with same configurations (depth, width, activation, dropout, optimizer, learning rate, loss), mini-batch training loop and learning-curve history as `Method_MLP.py`. Only the network itself is replaced. The following are subclasses each pertaining to one of the three CNN models:
  - `Method_CNN_MNIST.py`
  - `Method_CNN_ORL.py`
  - `Method_CNN_CIFAR.py`
- `Evaluate_Metrics.py` – accuracy and macro / weighted / micro precision, recall, F1; `Setting_Train_Test.py` – runs the pre-partitioned train/test experiment; `Result_Saver.py`, `Result_Loader.py`, `Plotter.py`.
- `script/stage_3_script` – directory of 3 scripts which train and evaluates the final 3 models;
  - `/script_cnn_mnist.py`
  - `/script_cnn_orl.py`
  - `/script_cnn_cifar.py`
- `script_ablation.py` – runs ablation experiments and writes the results table and plots.

3.5 Training Convergence Plot

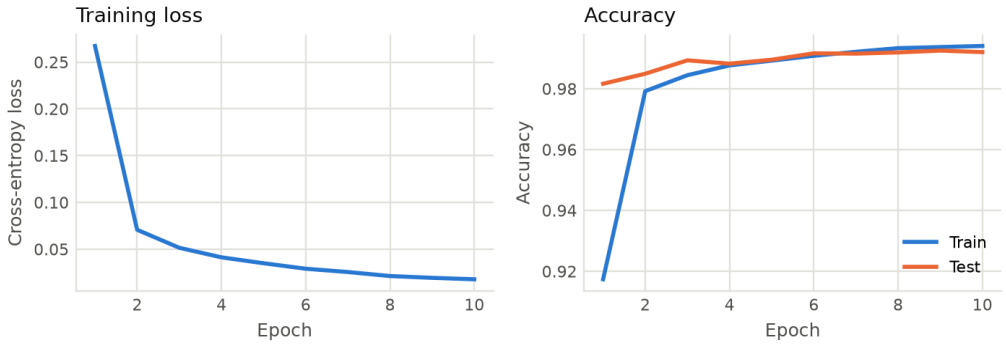


Figure 2: Training convergence of the final MNIST model: training loss (left) and train/test accuracy (right) per epoch.

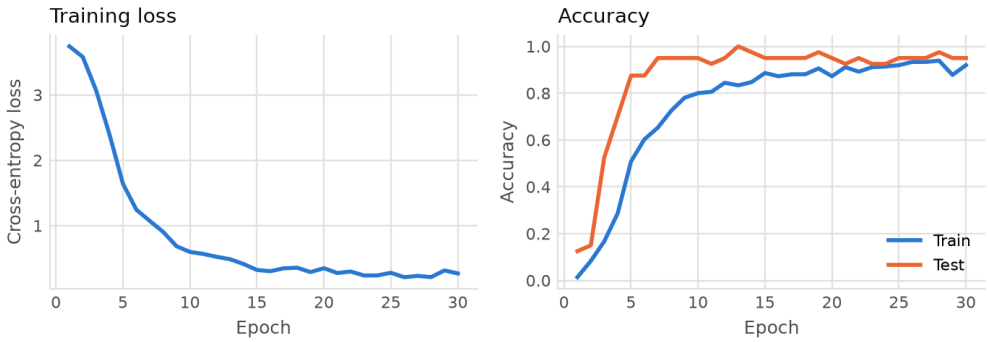


Figure 2: Training convergence of the final ORL model: training loss (left) and train/test accuracy (right) per epoch.

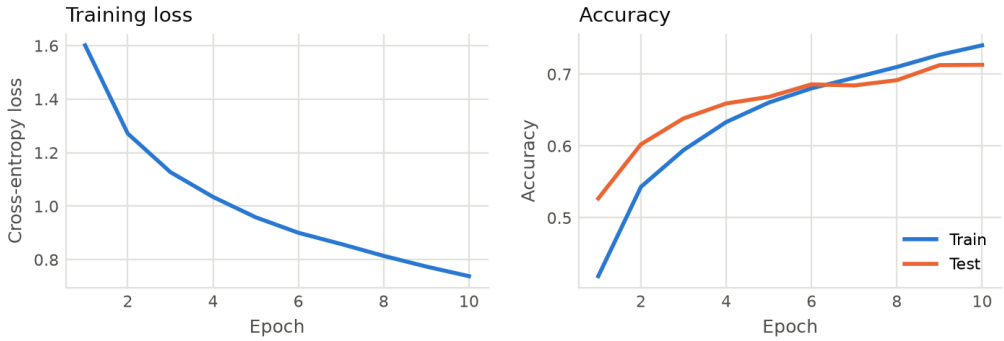


Figure 3: Training convergence of the final CIFAR model: training loss (left) and train/test accuracy (right) per epoch.

3.6 Model Performance

	MNIST			ORL			CIFAR		
Metric	**Macro**	**Weighted**	**Micro**	**Macro**	**Weighted**	**Micro**	**Macro**	**Weighted**	**Micro**
Precision	0.9920	0.9920	0.9920	0.9250	0.9250	0.9500	0.7139	0.7139	0.7129
Recall	0.9919	0.9920	0.9920	0.9500	0.9500	0.9500	0.7129	0.7129	0.7129
F1	0.9919	0.9920	0.9929	0.9333	0.9333	0.9500	0.7120	0.7121	0.7129
Accuracy	0.9920			0.9500			0.7129		

Table 2: Performance of the 3 final models on the testing sets.

3.7 Ablation Studies

We performed several ablation tests to measure how different CNN configurations (depth, kernel size, padding, stride, pooling, hidden dimensions, and loss function) affect performance against the MNIST, ORL, and CIFAR datasets. Each run tests the CNN with one setting changed from the baseline, keeping everything else fixed. Baseline settings are 2 convolution blocks, a 3x3 kernel, padding 1, stride 1, max pooling, 128 hidden dimensions, and the cross-entropy loss function. The baseline model differs across datasets with regard to dropout, batch size, and number of epochs. We recorded accuracy, precision, recall, and F1-scores for each setting change to a csv for each dataset (`result/stage_3_result/ablation_table_*.csv`). Table 3 displays accuracy of each configuration with three random seeds, reported as mean +- std (population std). Only results outside the seed spread should be considered real effects.

Factor	Setting Change	Accuracy		
		MNIST	**ORL**	**CIFAR**
Baseline Settings	2 conv blocks, 3x3 kernel, padding 1, stride 1, max pooling, hidden dim 128, cross-entropy loss	**0.9915 +- 0.0004**	**0.9583 +- 0.0118**	**0.7015 +- 0.0083**
Depth	1 conv block	0.9866 +- 0.0004	0.0333 +- 0.0118†	0.6415 +- 0.0075
Depth	3 conv blocks	0.9921 +- 0.0009	0.9667 +- 0.0118	0.7274 +- 0.0051
Kernel size	5x5	0.9926 +- 0.0013	0.9500 +- 0.0204	0.7071 +- 0.0058
Padding	padding 0	0.9919 +- 0.0003	0.9250 +- 0.0000	0.6776 +- 0.0100
Stride	stride 2	0.9864 +- 0.0009	0.9750 +- 0.0354	0.6028 +- 0.0073
Pooling	average	0.9911 +- 0.0002	0.9750 +- 0.0204	0.6645 +- 0.0050
Hidden dim	512	0.9912 +- 0.0005	0.9750 +- 0.0204	0.7332 +- 0.0040
Loss function	MSE on softmax	0.9907 +- 0.0012	0.9667 +- 0.0236	0.6986 +- 0.0040

Table 3: Accuracy of the CNN on the three datasets with several setting changes.
† 1 convolution block on ORL: *training failed to converge; not a reliable measure of depth*

Analysis:

1. Setting changes against the MNIST dataset caused effects which were mostly within noise. Only 1 convolution block and stride 2 are outside of the seed spread, both decreasing average accuracy by ~0.5 points.
2. The CIFAR dataset saw clear results with ablation to depth, padding, stride, pooling, and hidden dimensions. Accuracy rose with depth (64→70→73 points) as well as with more hidden dimensions (70 points at 128 vs. 73 points at 512). Accuracy decreased with stride 2 by 9.9 points, with average pooling by 3.7 points, and with padding by 2.4 points. Effects of changes in kernel size and MSE loss function were within noise.
3. The model scored roughly 2 points higher on the ORL dataset with average pooling, 512 hidden dimensions, and stride 2. Accuracy decreased by about 3 points with padding 0. However, with a testing set of 40 images, each test is worth 2.5 points before averaging between seeds. Therefore, differences between settings represent roughly one test image per run and are not distinguishable. The 1-conv-block run failed to converge (loss stayed at ln 40 ≈ 3.69), so it says nothing about depth.
4. The effect of increasing from stride 1 to stride 2 varies across datasets. It was among the highest recorded accuracy, within noise, on the ORL dataset and garnered the lowest recorded accuracy for both the MNIST and CIFAR datasets (decreases of ~0.5 points and ~9.9 points from baseline, respectively). This may be due to the difference in image size, as the ORL images are significantly larger than the MNIST and CIFAR images. A larger stride is likely hurtful to performance of the CNN on smaller images because it throws away spatial details that are essential for classification.